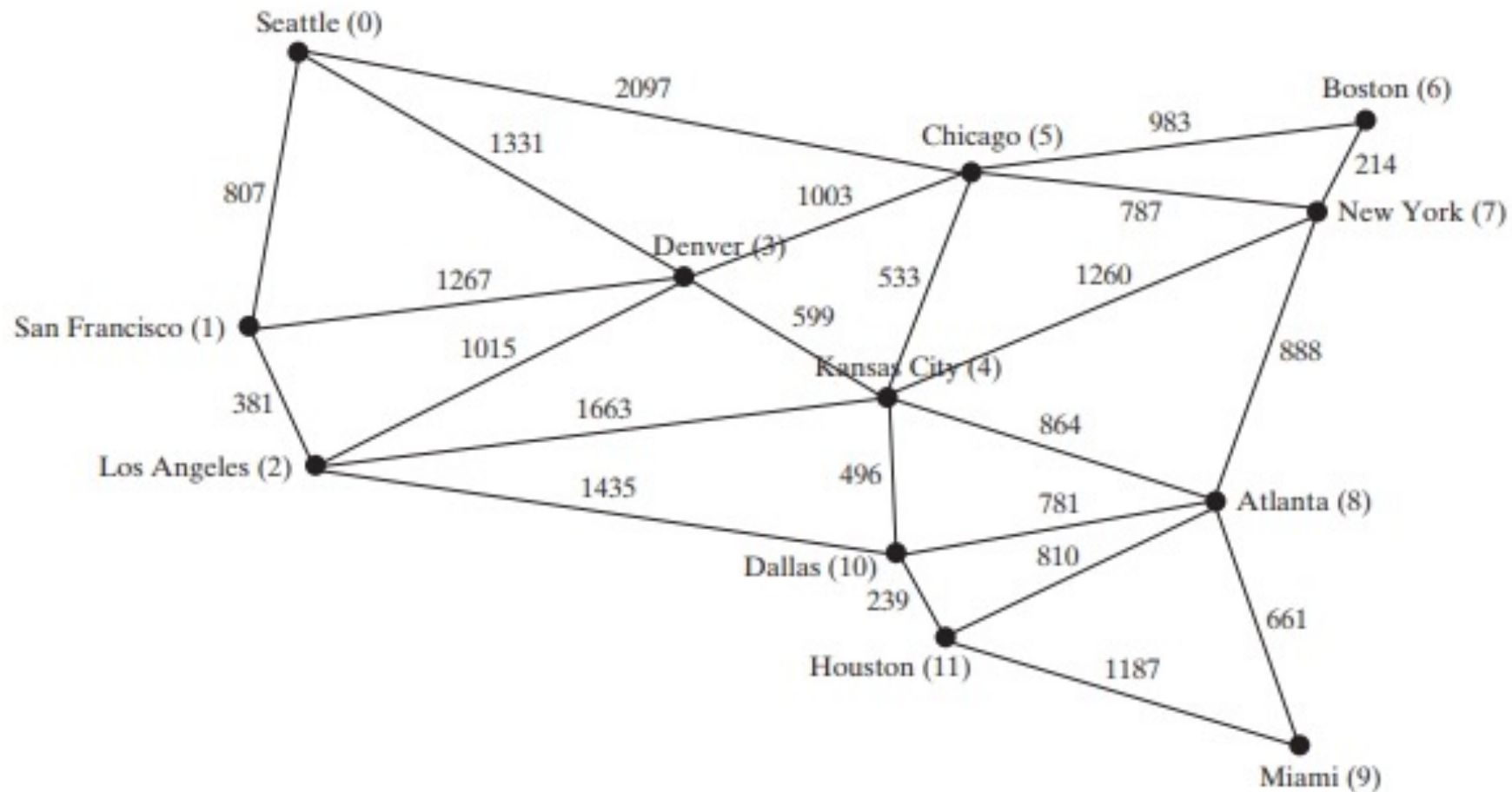




Shortest Paths

Warm-Up

- Take 5 minutes, determine the shortest path from SF to Miami.
What algorithm did you use?



Dijkstra's Shortest Path

- Further watching:
https://youtu.be/_IHSawdgXpI?si=5Y13a9QefeYVZOVM
- Goal: find shortest distance from one node to every other node.
- Have a chart of distances from start node
- At each step, update table for any new distances (choose smallest possible)
- Result: Shortest Path Tree
- Complexity: $O(|E| + |V| \log |V|)$

Dijkstra's Algorithm

dijkstra(v):

 pq = new PriorityQueue()

 pq.insert([dest:v, pred:null, cost:0])

 while (!pq.isEmpty()):

 [dest, pred, cost] = pq.removeMin()

 if dest is unvisited:

 mark dest as visited, store pred and cost for dest

 for each edge with weight w to unvisited neighbor u of dest:

 pq.insert([u, dest, cost + w])

dijkstra(v):

```
pq = new PriorityQueue()
```

```
pq.insert( [ dest:v, pred:null, cost:0 ] )
```

```
while ( !pq.isEmpty() ):
```

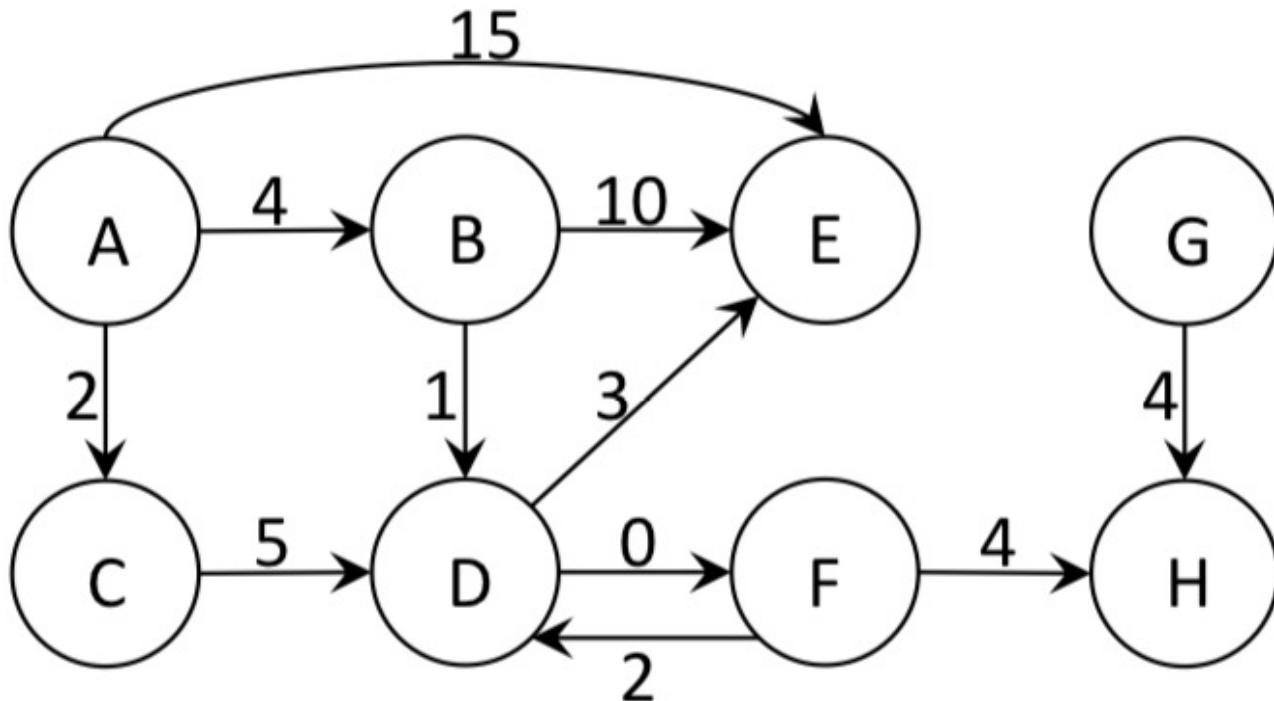
```
    [ dest, pred, cost ] = pq.removeMin()
```

```
    if dest is unvisited:
```

```
        mark dest as visited, store pred and cost for dest
```

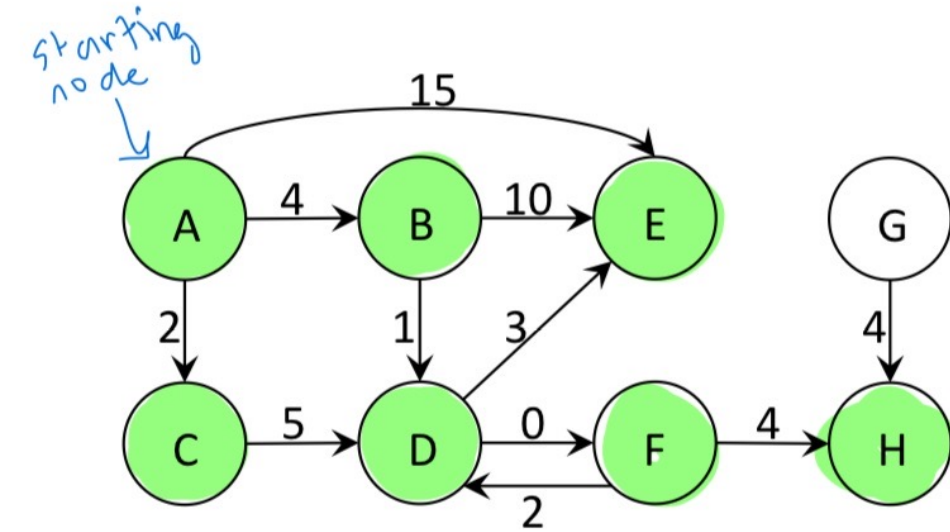
```
        for each edge with weight w to unvisited neighbor u of dest:
```

```
            pq.insert( [ u, dest, cost + w ] )
```



Vertex	Visited?	Pred.	Cost
A	no	n/a	
B	no		
C	no		
D	no		
E	no		
F	no		
G	no		
H	no		

Example



vertex	visited	pred	cost
A	F T	null	0
B	F T	A	4
C	F T	A	2
D	F T	B	5
E	F T	D	8
F	F T	D	5
G	F		
H	F T	F	9

priority queue: ~~[A, null, 0]~~, ~~[B, A, 0+4]~~, ~~[C, A, 0+2]~~, ~~[E, A, 0+15]~~, ~~[D, C, 2+5]~~, ~~[D, B, 5]~~, ~~[E, B, 14]~~,
~~[E, D, 8]~~, ~~[F, D, 5]~~, ~~[H, F, 9]~~

visited already

visited already

visited already

Let's try it again

dijkstra(v):

```
pq = new PriorityQueue()
```

```
pq.insert( [ dest:v, pred:null, cost:0 ] )
```

```
while ( !pq.isEmpty() ):
```

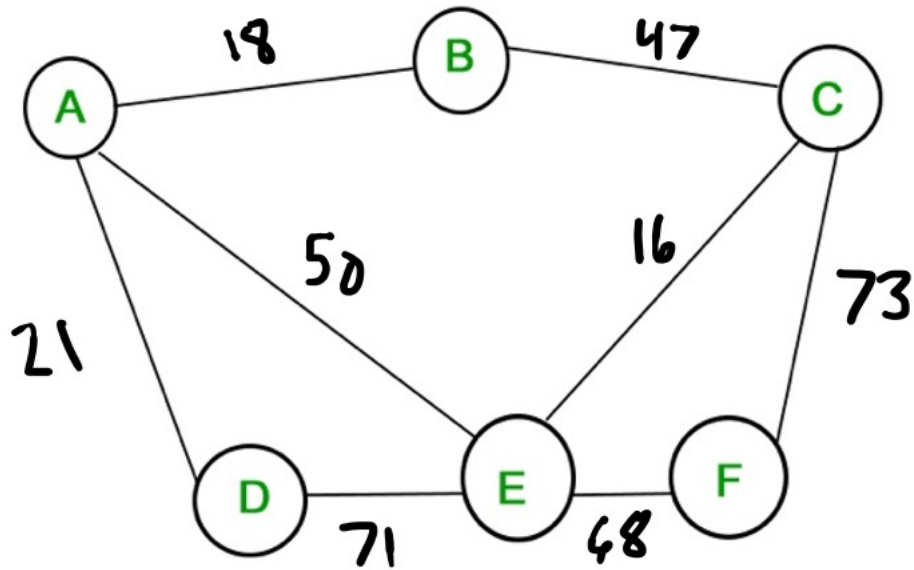
```
    [ dest, pred, cost ] = pq.removeMin()
```

```
    if dest is unvisited:
```

```
        mark dest as visited, store pred and cost for dest
```

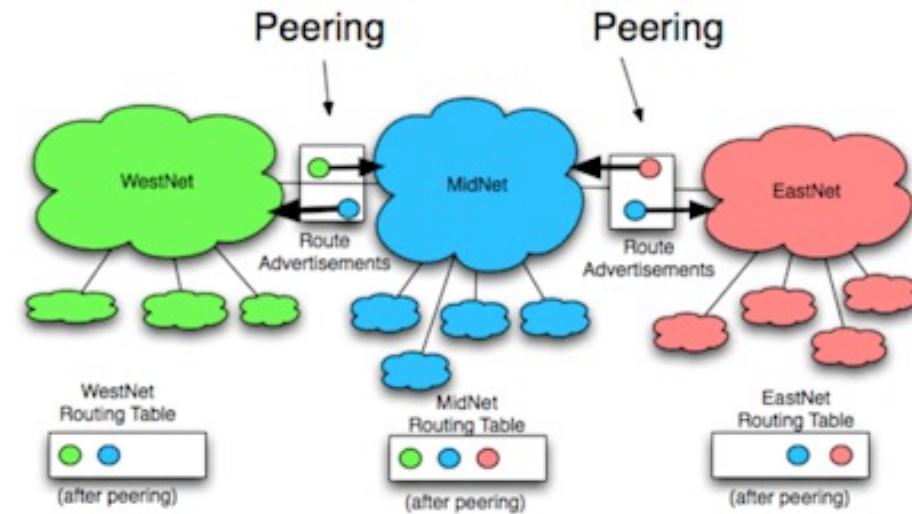
```
        for each edge with weight w to unvisited neighbor u of dest:
```

```
            pq.insert( [ u, dest, cost + w ] )
```



Vertex	Visited?	Pred.	Cost
A	no		
B	no		
C	no		
D	no		
E	no		
F	no		

How is Dijkstra's used?



- In general: Finding shortest path between two nodes
 - Your favorite Maps app!
- Networking:
 - [Border Gateway Protocol](#) in networking:
 - How to move this packet to another router so it gets where it needs to go? Each router has a lookup table.

Other shortest path algorithms

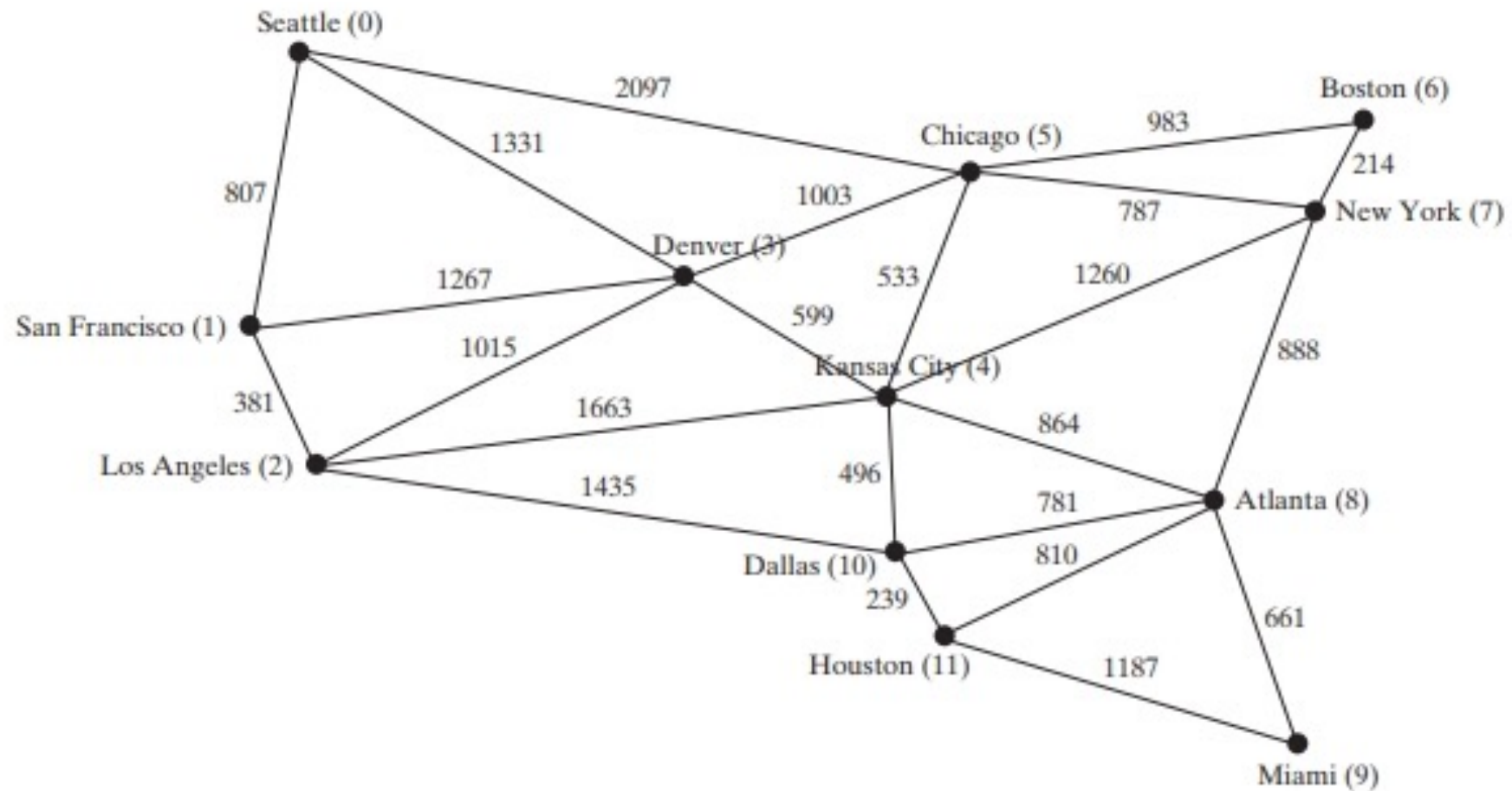
- Bellman-Ford $\Theta(|V||E|)$
 - Slower than Dijkstra, but can handle negative edge weights
- Floyd-Warshall - $O(|V|^3)$
 - Also can handle negative edge weights, but it's cubic.

YOU TRY

1. Choose one of your sample graphs
2. Go to the board and run Dijkstra's algorithm
3. Highlight the shortest path tree
4. Discuss: How is the "shortest path tree" different from the "minimum spanning tree"?

YOU TRY

Shortest distance from SF to Miami?

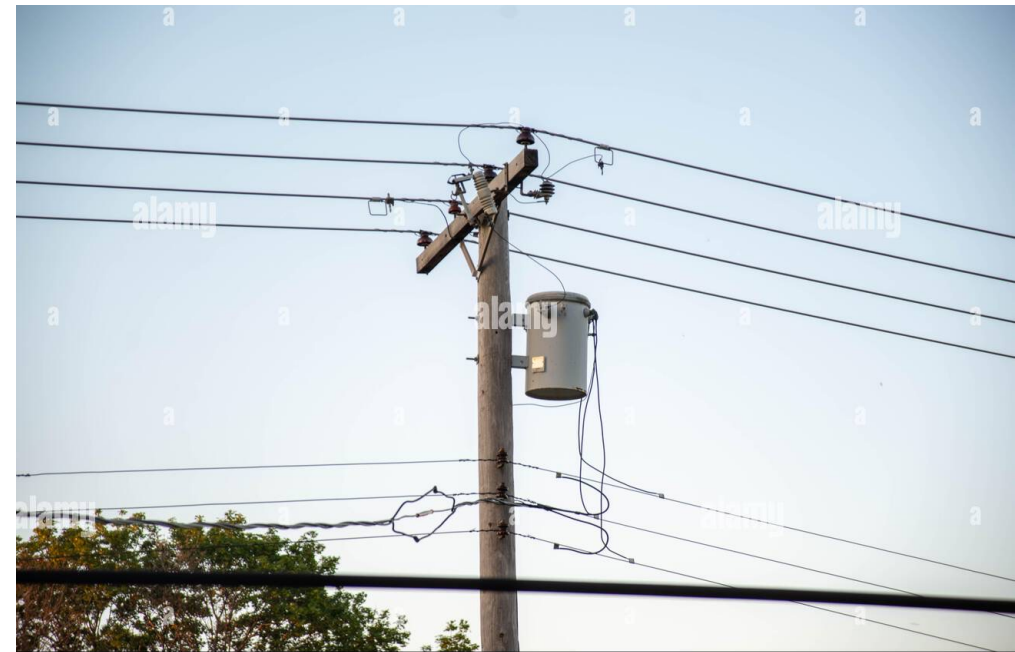
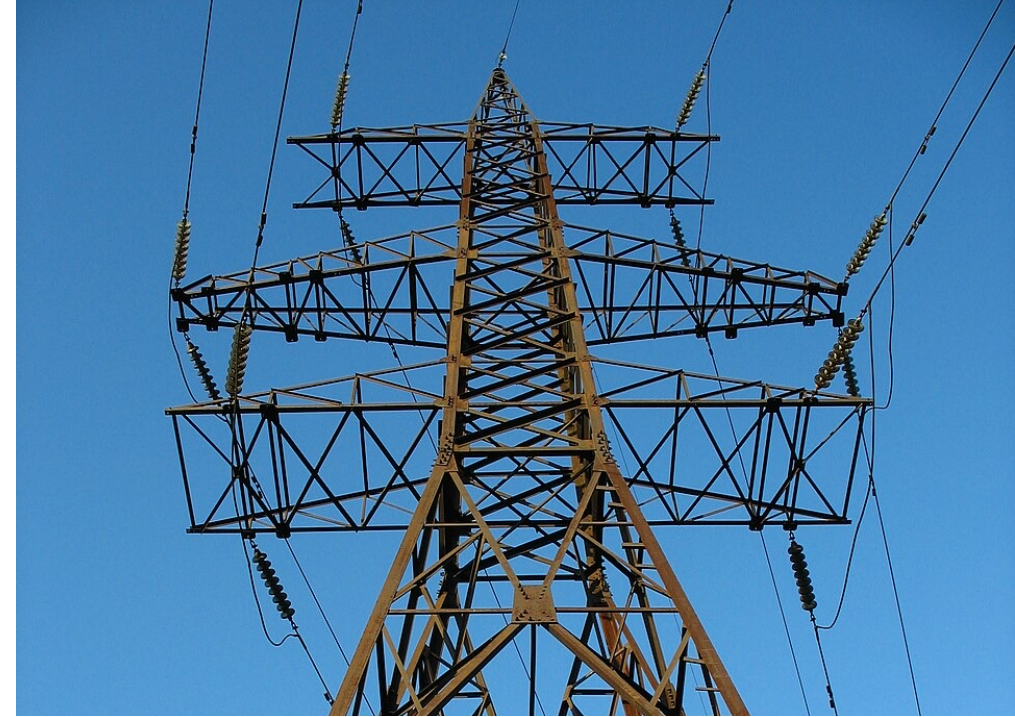




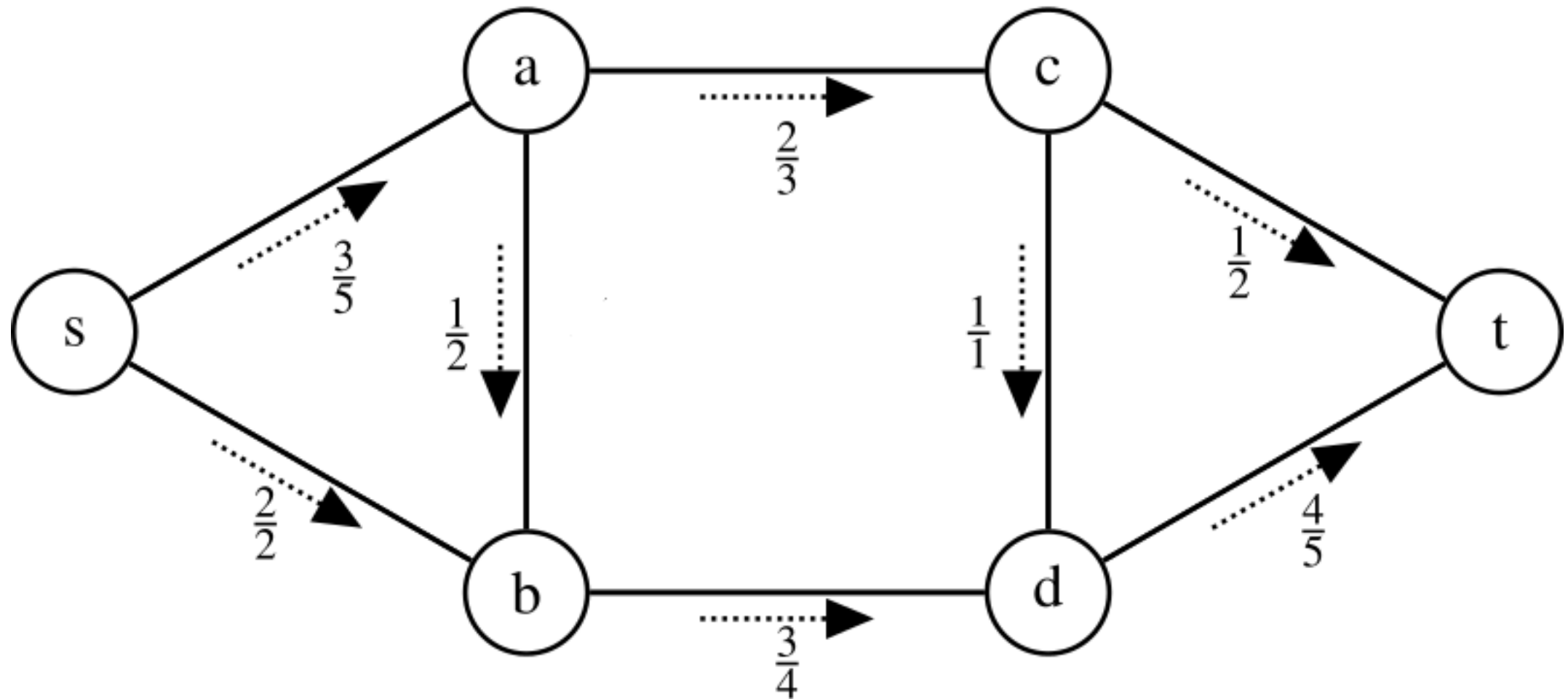
Flow Network

Motivation

- For your PG&E internship, you design the electrical network for a small town.
- They have few power lines, but need their town hall to have power.
- Each line is rated for a certain capacity.
- How much current can the town hall receive?



The community's electrical grid



What is a flow network?

- Used to model the passage of "material" throughout a network.
- From s to t
 - Source to Target
 - Source to *Sink*
- Each edge's weight represents its *capacity* (non-negative)
- We also denote the edge's *flow*, i.e. how much material goes through that edge.
- We will write: flow / capacity

Limitations

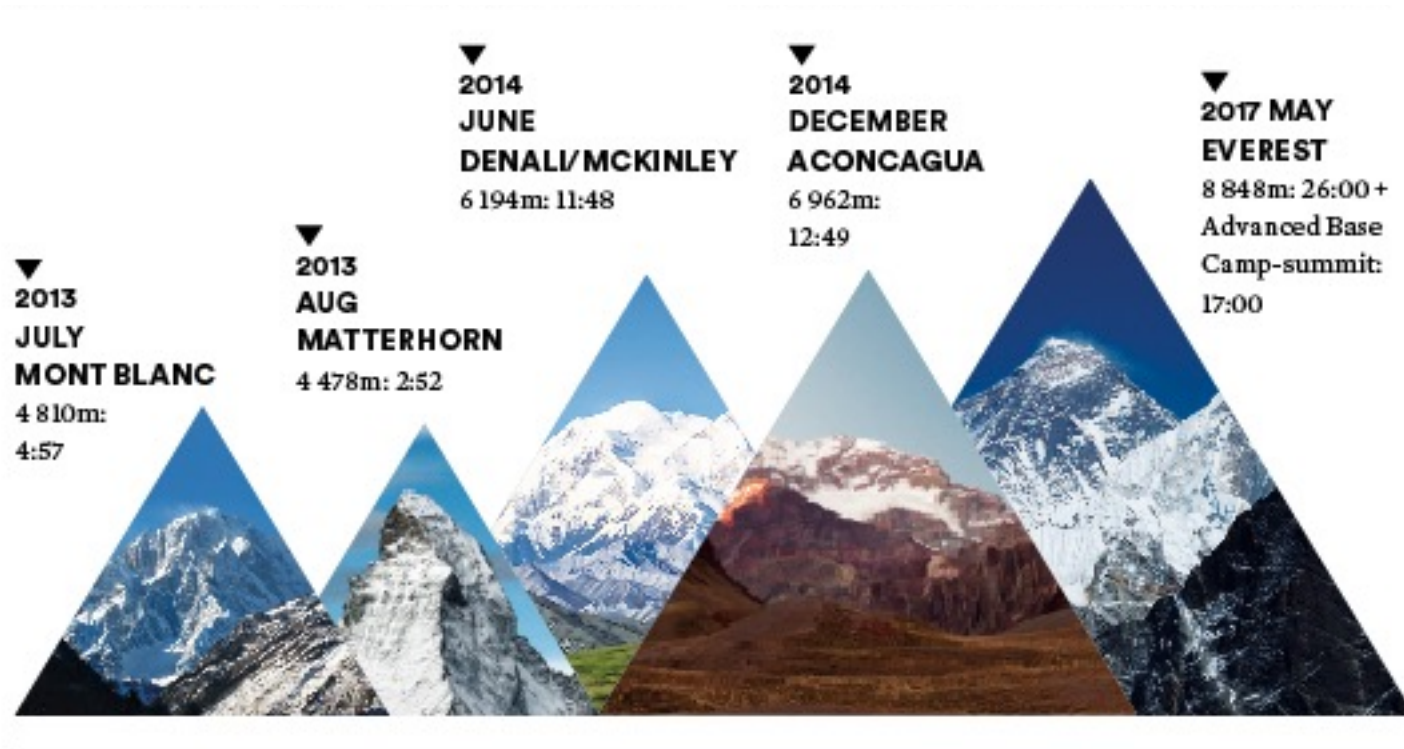
- What happens if we overload the power line?
 - Flow must be $\leq$ capacity
- What does it mean for a line to have negative capacity?
 - Capacity & flows must be non-negative integers

Questions we can answer

- How much current can we send from the transmission tower to town hall?
 - *Maximal flow*

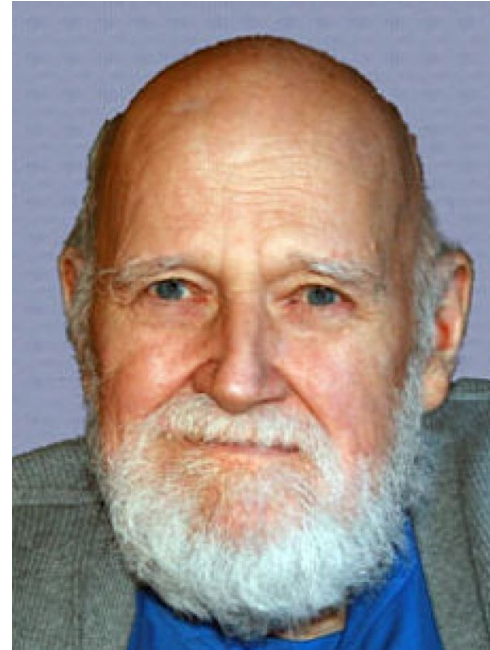
Tangent: maximal vs. maximum

- Maximal = highest possible in the current situation.
- Maximum = highest possible overall.

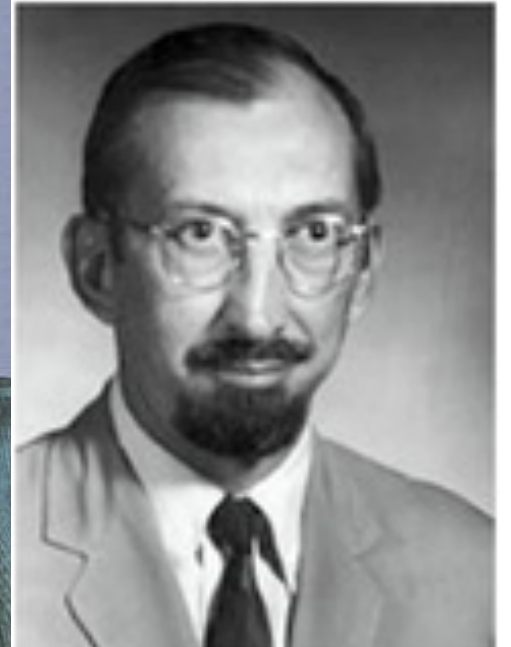


FORD-FULKERSON ALGORITHM FOR MAXIMUM FLOW

Complexity: $O(Ef)$ where f is the maximum flow



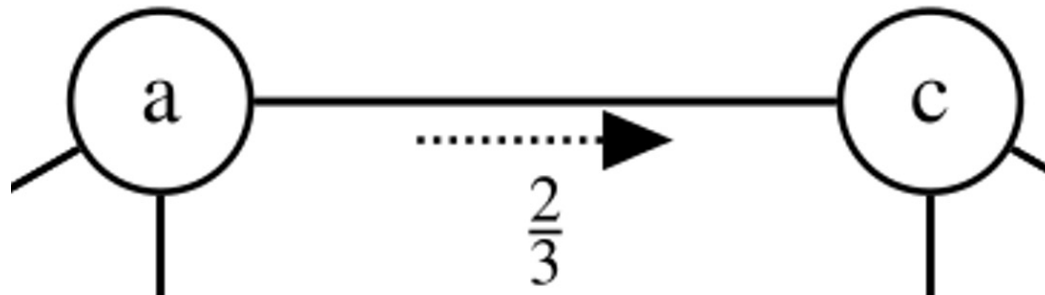
Ford



Fulkerson

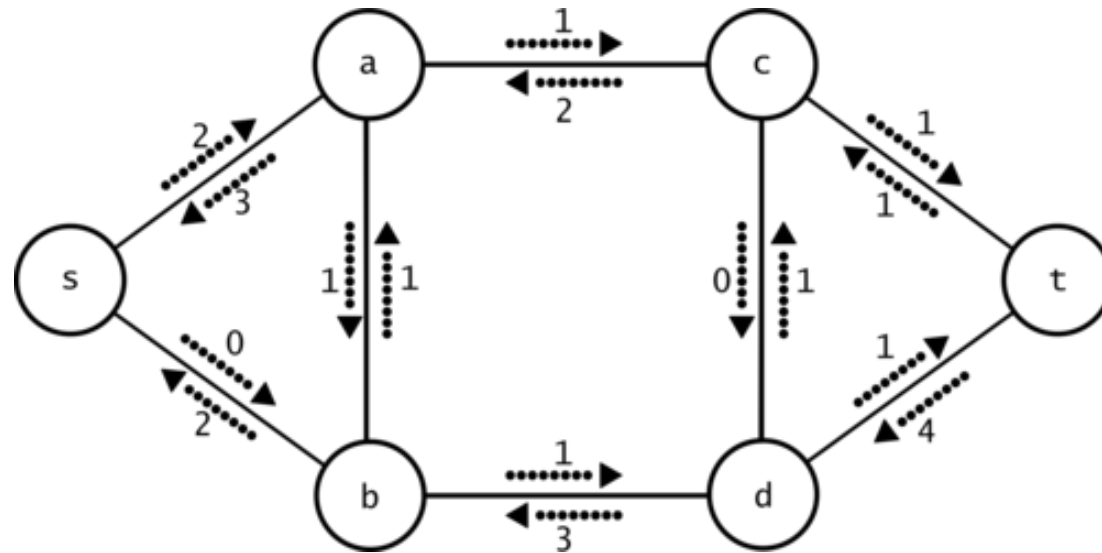
Residual Capacity

- Capacity = stated amount the edge can handle
- Flow = amount moved through the edge
- Residual capacity = anything left over
- If flow = capacity: the edge is *saturated*
- Ex. Residual capacity of ac is $1 = 3 - 2$. ac is not yet saturated.



Showing Residual Capacity

- Draw an arrow going backwards / write it on the graph

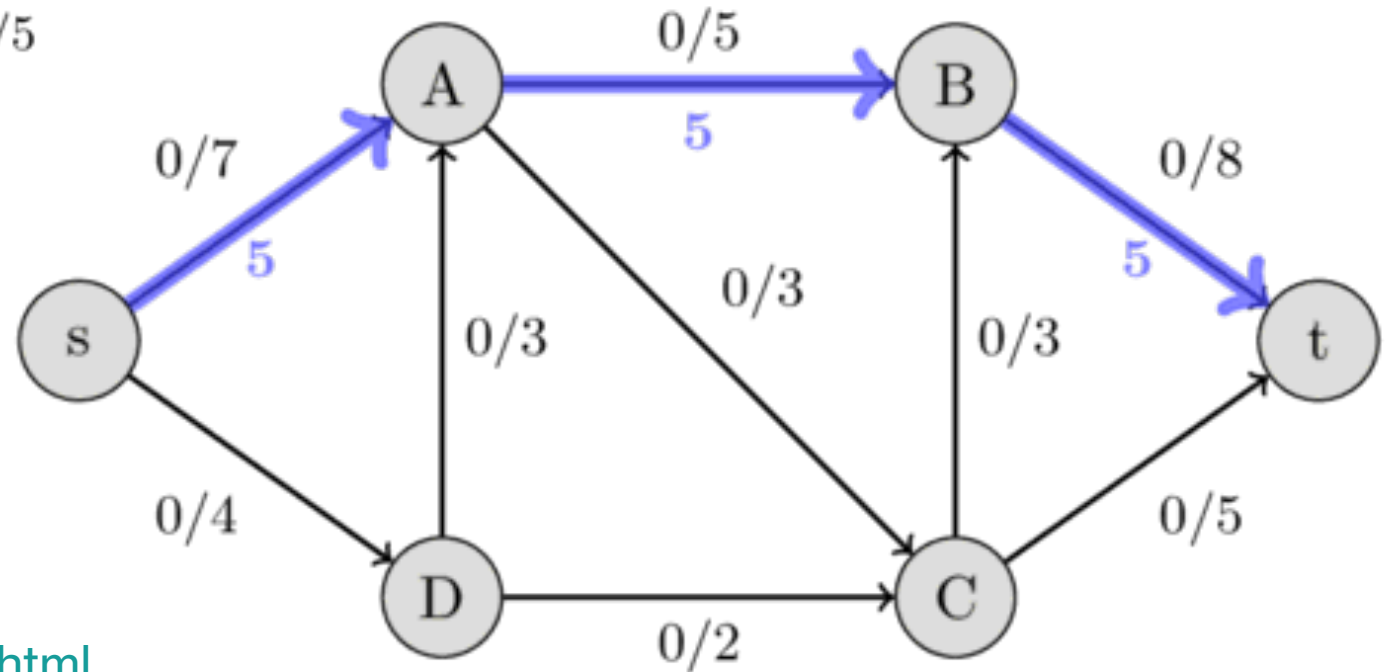
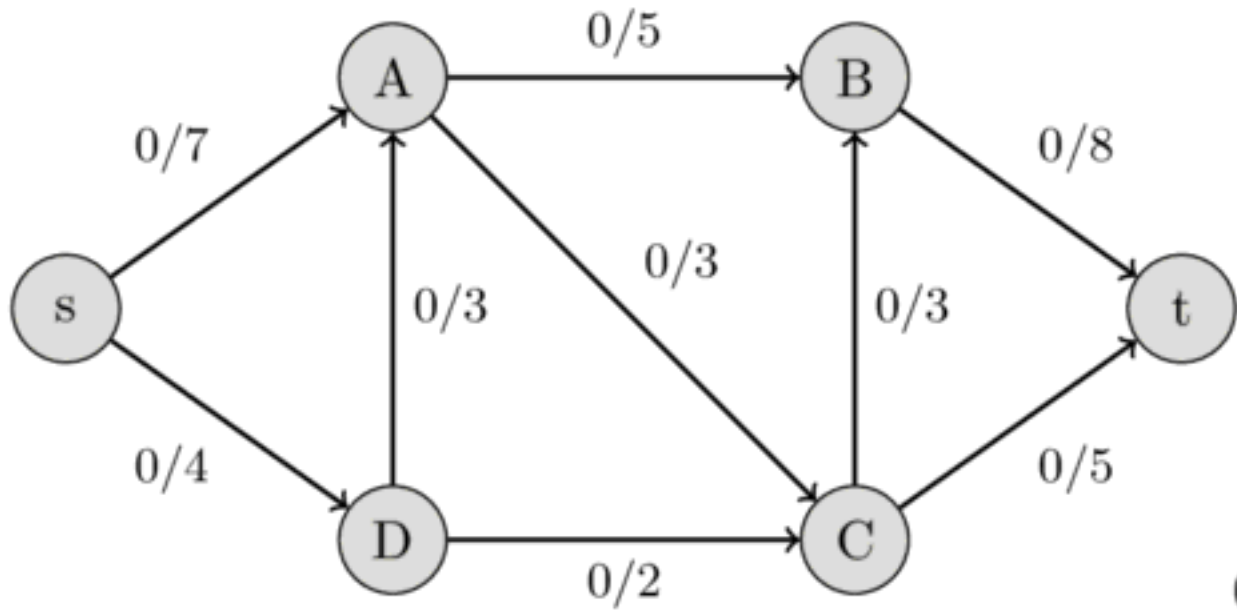


- Or keep it in your head, if it's simple to do the math

Augmenting Path

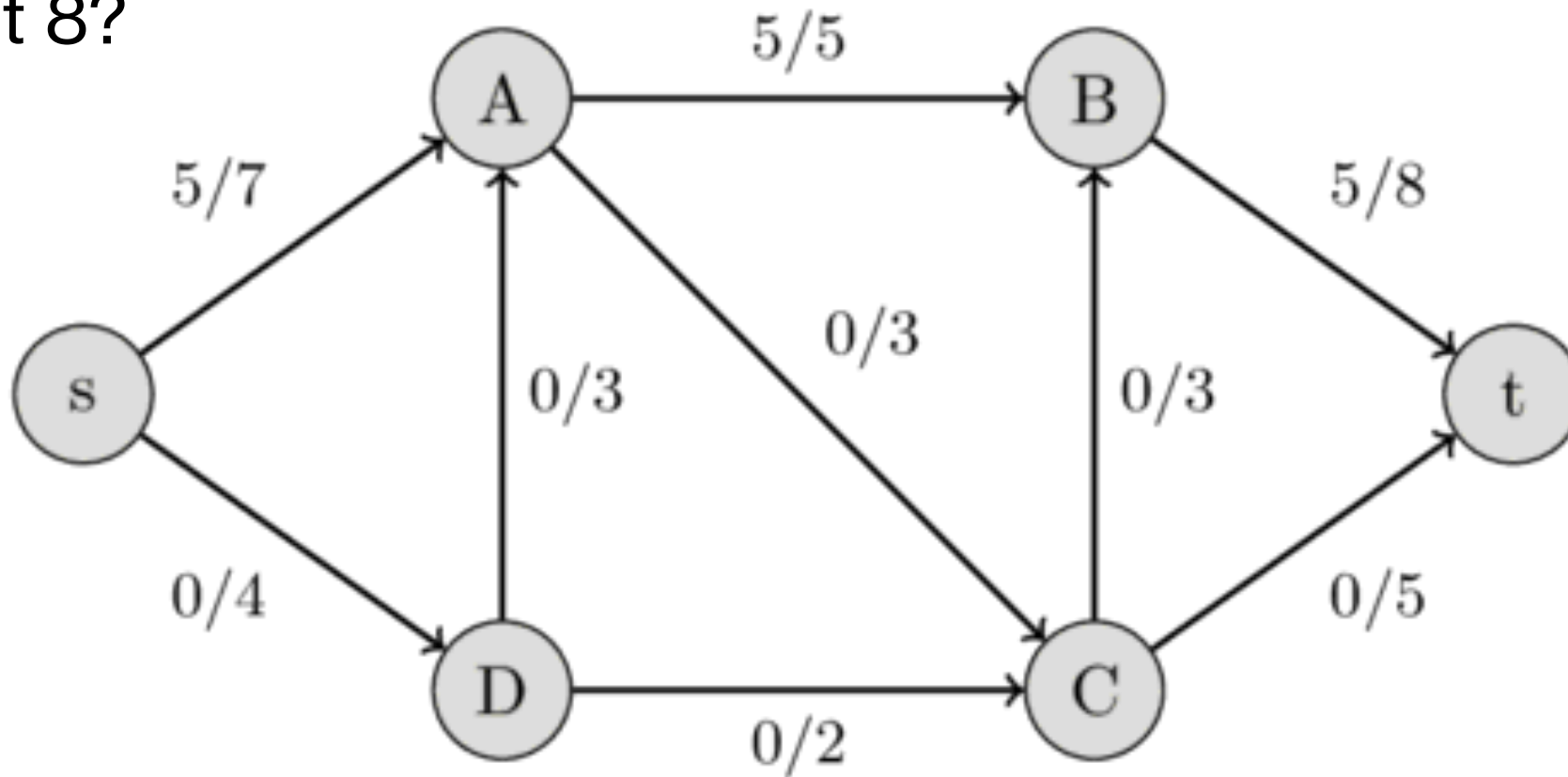
- A *simple* path where residual capacity is positive
 - Simple = no repeating vertices
 - Found augmenting path? We can increase the flow of that path.
 - How much to increase flow? By the minimum (remaining) capacity of that path.

Augmenting path from s to t



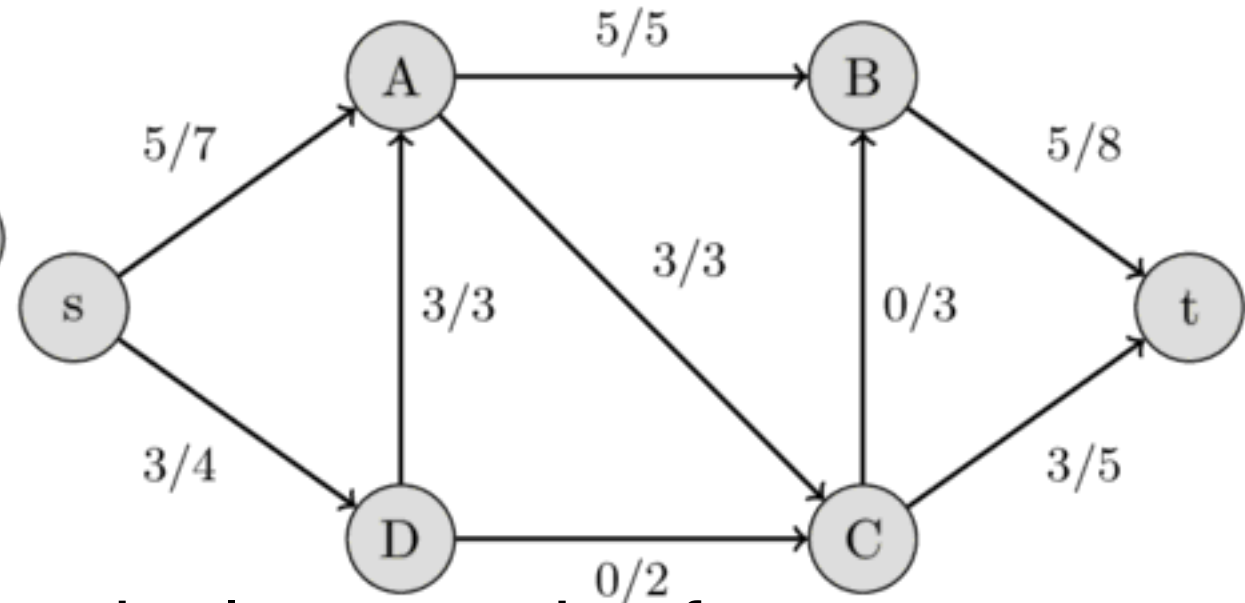
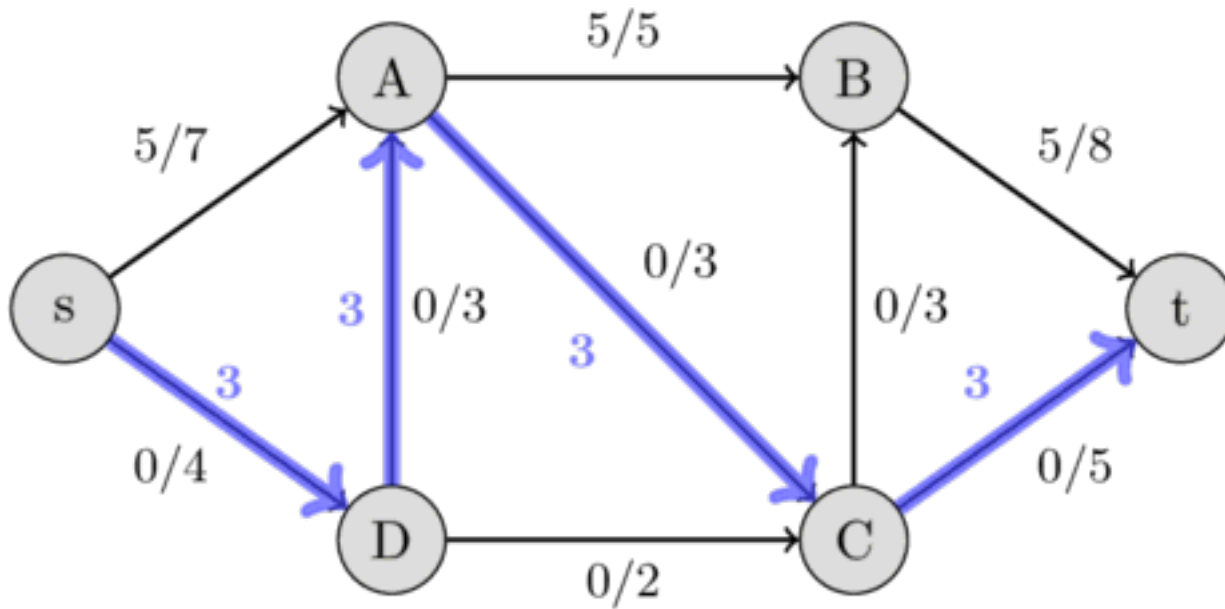
Resulting flow network

- Check for understanding: Why does $B \rightarrow t$ have a flow of 5, and not 8?



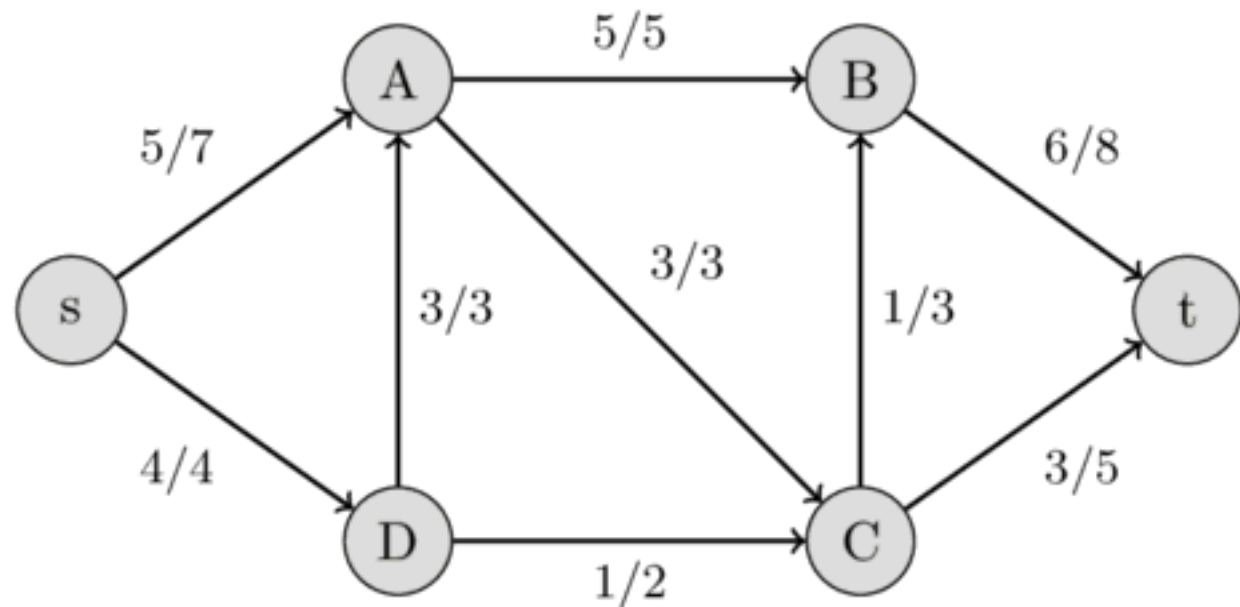
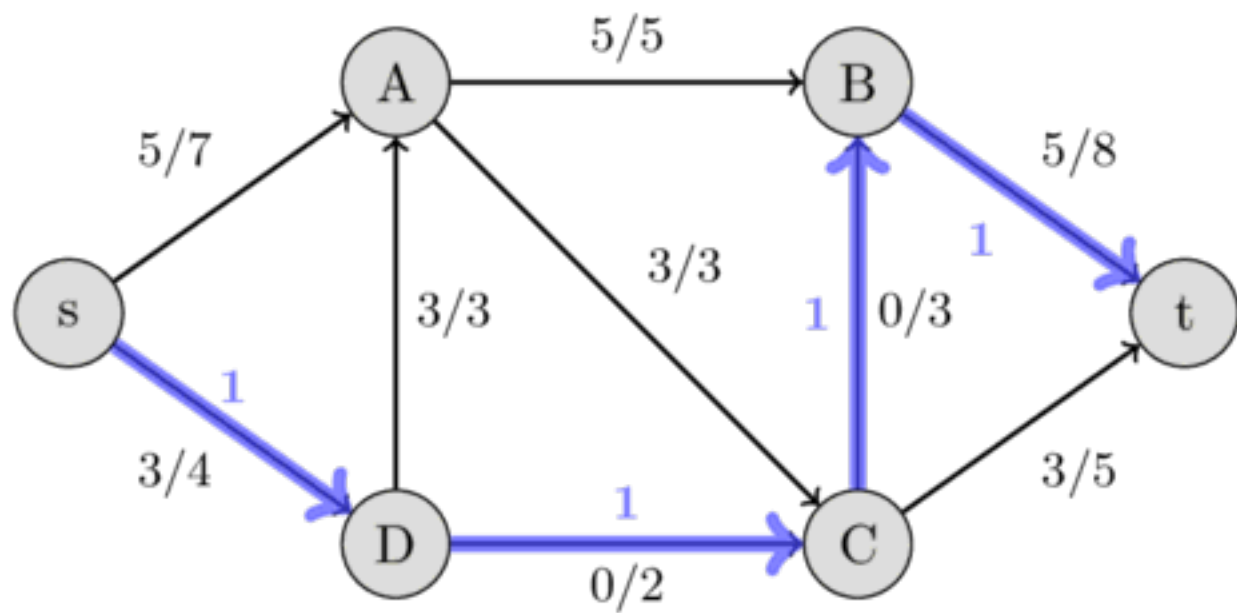
Let's do it again

- Find another augmenting path
 - Where residual capacity is positive



- Not worrying right now about *how*, that's a question for another time

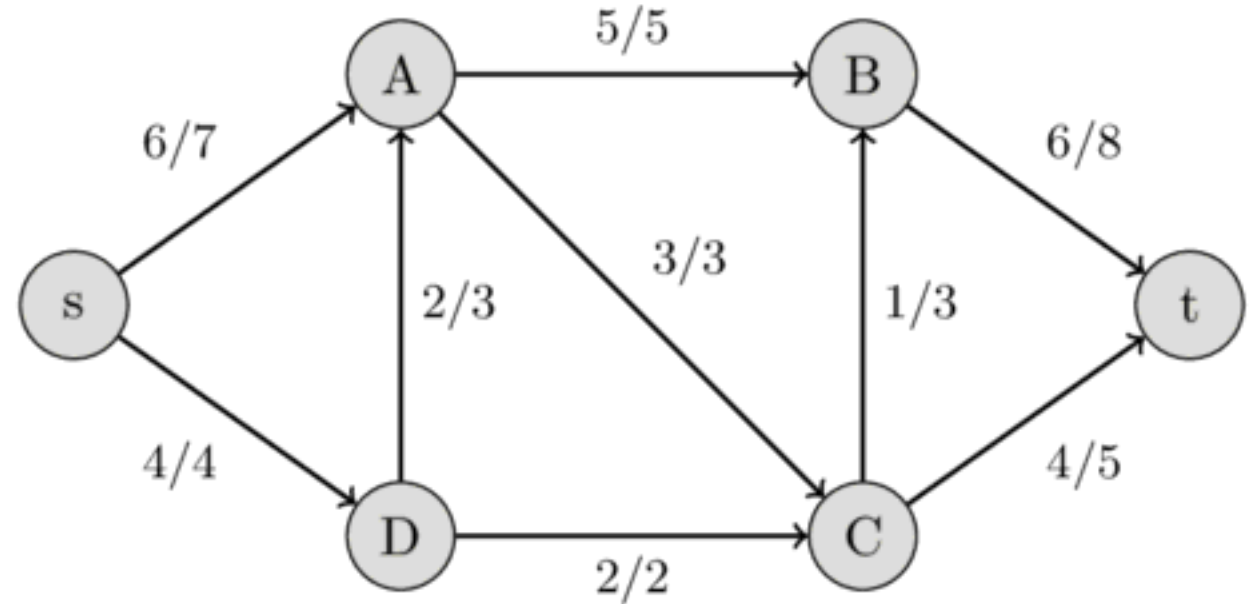
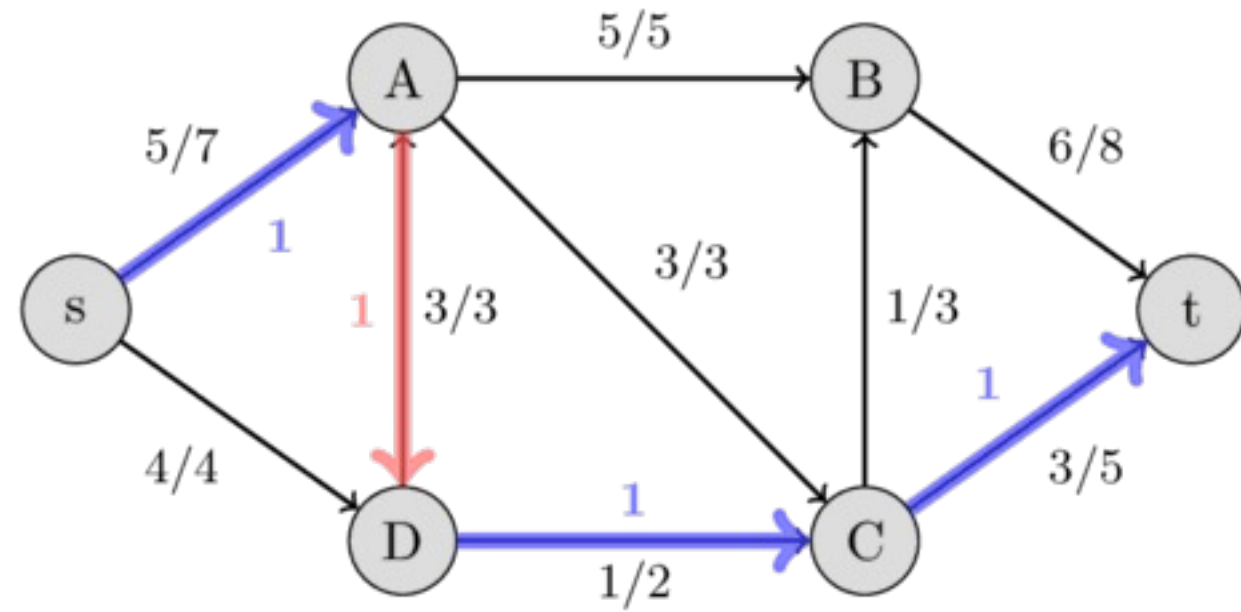
Let's do it again



As we go

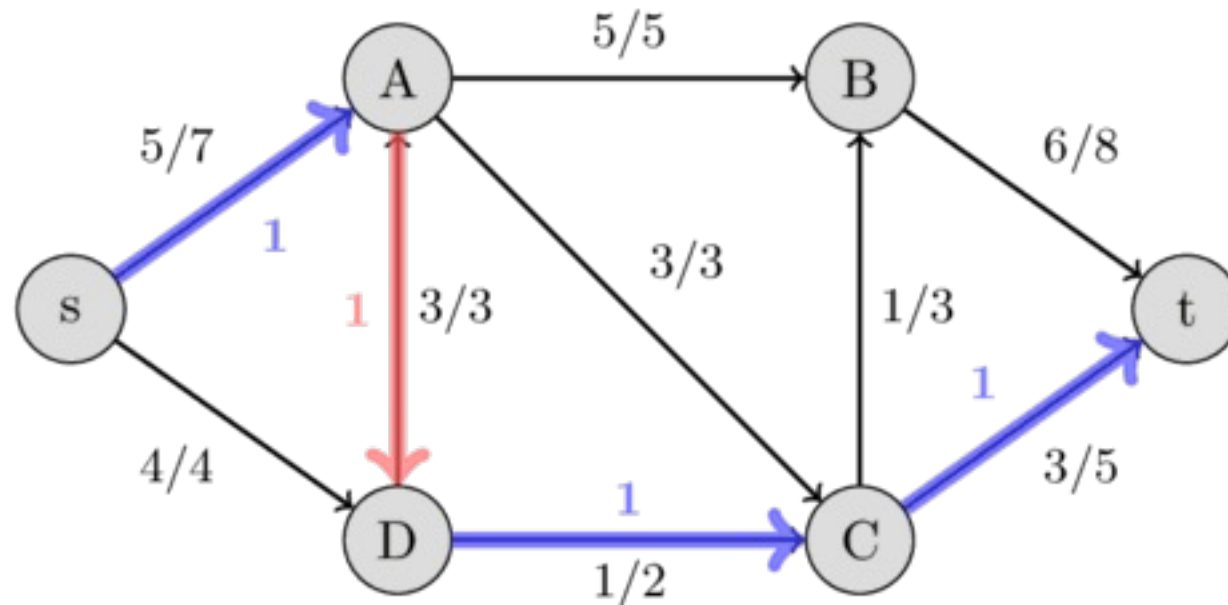
- When an edge's flow = capacity, the edge is *saturated*.
- Each vertex must be in equilibrium
- When looking for augmenting path, we can:
 - Choose a "forward" edge if it has capacity to spare
 - Choose a "backwards" edge if it has non-empty flow

And again



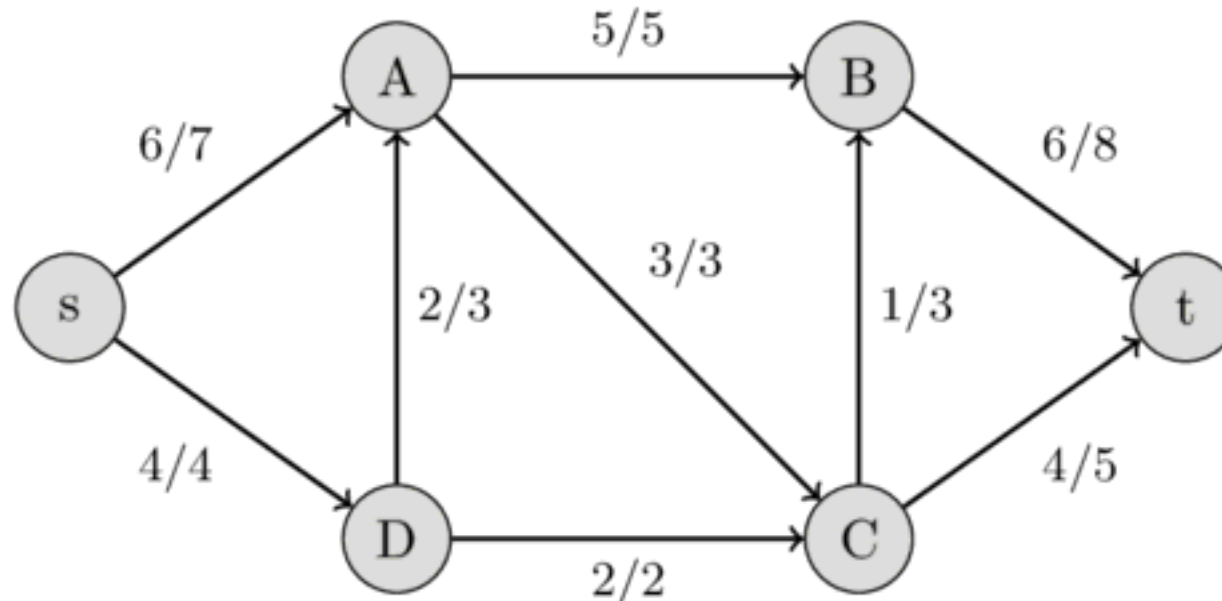
Why does this work?

- Instead of sending everything through $D \rightarrow A$, don't send one unit, and send it through $D \rightarrow C$ instead.
- Note that $\text{in}(A)$ and $\text{out}(A)$ is still 5. A is still in equilibrium.



Our result

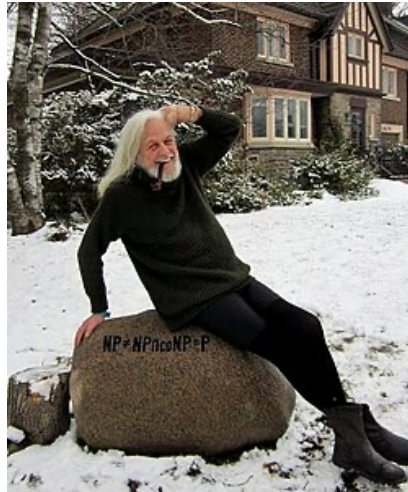
- Once we cannot find an augmenting path, we are done.
 - $s \rightarrow D$ is saturated; $s \rightarrow A$ has space, but $A \rightarrow B$ & $A \rightarrow C$ don't, nor does $D \rightarrow C$ even if we reversed flow in $A \rightarrow D$.
- Max flow is $\text{out}(s) = \text{in}(t) = 6 + 4 = 10$.



EDMONDS-KARP ALGORITHM

Complexity:

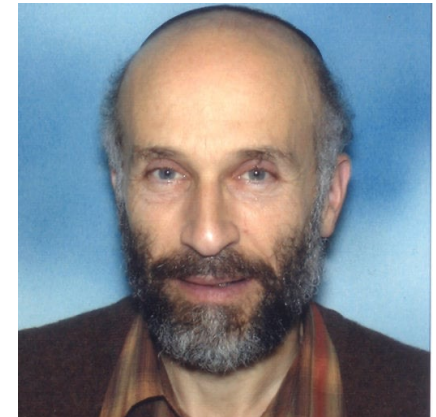
$O(VE^2)$ (E-K) or $O(V^2E)$ (Dinitz)



Jack Edmonds



Richard Karp

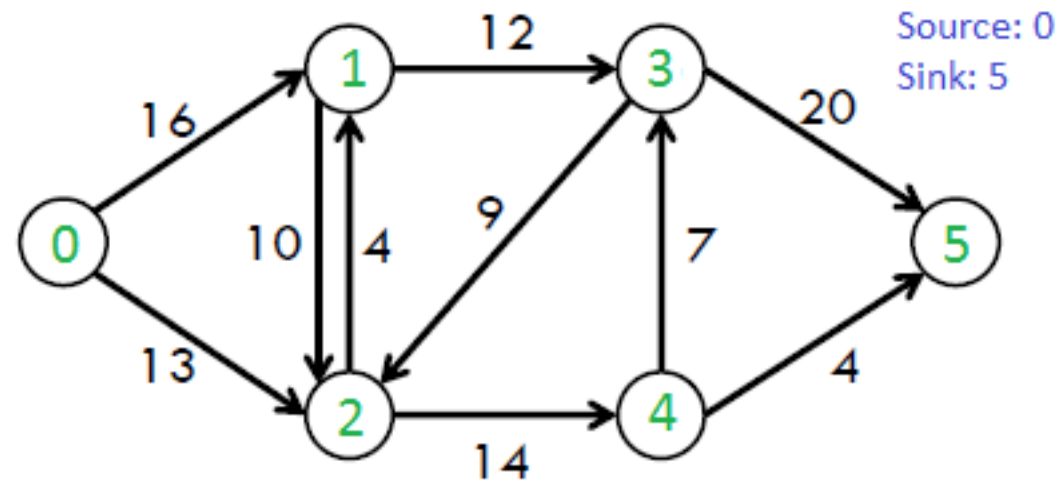


Yefim Dinitz
(published independently)

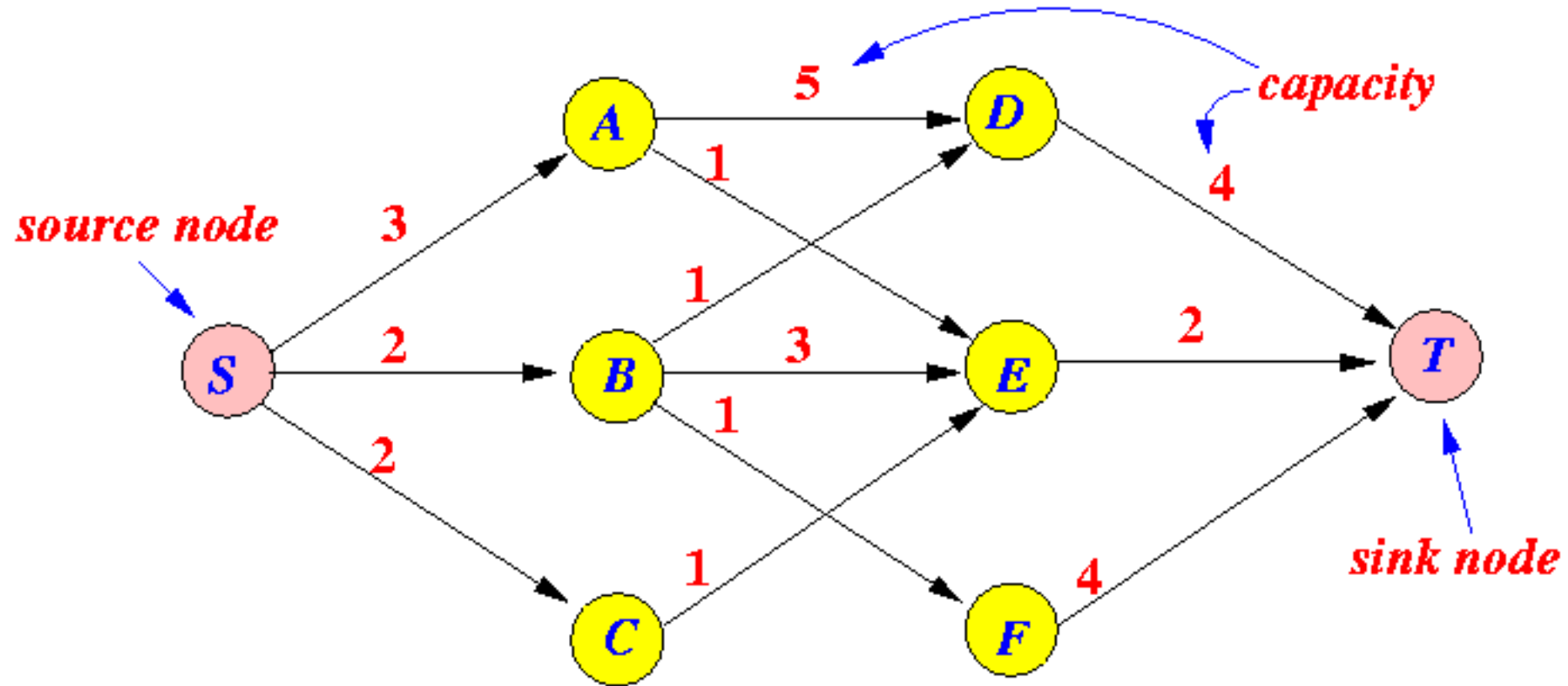
Key distinction

- When finding an augmenting path, determine the *shortest path* with *available capacity*.
- Use BFS or DFS greedily

Let's try



YOU TRY



I think the answer is 5; prove me right or prove me wrong.

Cool-down

- An airline wishes to know how many crews it should allocate for the following airport map. Assume we want to send crews from Vancouver to Winnipeg.

